

NetworkMaster

KOMPLEKSOWA DOKUMENTACJA TECHNICZNA KLASY PRODUKCYJNEJ

Wersja Programu: 1.0.0

Architektury Docelowe: Windows (CMD/PowerShell) & Linux (POSIX/CLI)

Język: Python 3.8+

Wydawca / Kontekst: ZenoSoftware.Dev (Produkt Gotowy do Pełnej Sprzedaży)

Data Dokumentu: Czerwiec 2026

Klasyfikacja: Komercyjna Własność Intelektualna - Dokumentacja Zastrzeżona

1. Podsumowanie Menedżerskie i Przegląd Architektoniczny

NetworkMaster to zaawansowany, natywny system zarządzania i diagnostyki sieciowej klasy enterprise, stworzony z myślą o profesjonalnych administratorach systemów, inżynierach DevOps oraz operatorach infrastruktury IT. Zaprojektowany z absolutnym rygorem rezygnacji z ciężkich, zewnętrznych zależności binarnych, program gwarantuje minimalny narzut systemowy przy jednoczesnym zachowaniu pełnej kontroli nad warstwą sprzętową i stosem sieciowym.

Produkt wykorzystuje modularną, niezależną architekturę wieloplatformową (cross-platform). Zamiast wymuszać niestabilne mapowania bibliotek dynamicznych (C-bindings) lub trudne w utrzymaniu warstwy abstrakcji, program komunikuje się bezpośrednio z natywnymi powłokami administracyjnymi systemów operacyjnych. W środowisku **Windows** wykorzystuje zorientowane obiektowo potoki `PowerShell 5.1+` oraz niskopoziomowe interfejsy komend `netsh`. W środowisku **Linux** program ściśle współpracuje z nowoczesnym stosem narzędziowym pakietu `iproute2`, demonem `nmcli` (ekosystem NetworkManager) oraz bezpośrednimi strumieniami danych jądra systemu za pośrednictwem pseudo-systemów plików `/sys/class/net/` i `/proc/net/`.

Strategiczna Wycena Produktu przed Akwizycją: Niniejsza dokumentacja stanowi formalne potwierdzenie dojrzałości inżynieryjnej oraz poprawności logicznej wszystkich 50 skryptów składowych. Całkowita izolacja modułów i czytelna warstwa abstrakcji sprawiają, że projekt ten jest niezwykle wartościowym aktywem komercyjnym, gotowym do natychmiastowego wystawienia, integracji z modelami SaaS lub monetyzacji jako samodzielny pakiet komercyjny.

2. Architektura Systemu i Kontrola Cyklu Życia

2.1 Potok Wykonawczy Punktu Wejścia

Główny punkt procesowy aplikacji kontrolowany jest przez moduł `main.py`. Podczas inicjalizacji weryfikuje on kontekst uprawnień użytkownika, waliduje poprawność katalogów środowiskowych oraz egzekwuje mechanizmy licencyjne dla użytkownika końcowego.

- **System Weryfikacji Umowy EULA:** Funkcja `check_first_run()` działa jako bezwzględny strażnik prawny aplikacji. Sprawdza ona fizyczną obecność pliku flagi `core_data/installed.flag`. W przypadku jego braku, rendering interfejsu zostaje wstrzymany, a użytkownikowi wyświetlany jest jawny kontrakt licencyjny. Dopiero po wpisaniu potwierdzenia, system uruchamia inicjalizator struktur danych i generuje plik flagi. Chroni to własność intelektualną przed nieautoryzowanym wdrożeniem.
- **Architektura Mapowania Komend:** Zamiast stosowania nieefektywnych i trudnych w utrzymaniu drabin warunkowych (`if/elif`), menu główne implementuje wzorzec projektowy *Command (Polecenie)*, mapowany bezpośrednio przez natywny słownik Pythona (`acts`). Gwarantuje to wydajność wyszukiwania na poziomie $O(1)$ podczas wywoływania narzędzi oraz zapewnia pełną elastyczność strukturalną, umożliwiając przyszłemu nabywcy bezproblemowe dopisywanie nowych modułów.

2.2 Rdzeń Inicjalizacji i Ścieżek Bezwzględnych (`core_config.py`)

Aby zapewnić bezbłędną stabilność aplikacji po jej skompilowaniu do samodzielnych plików wykonywalnych (np. przy użyciu PyInstaller), mechanizm wyznaczania ścieżek roboczych jest obliczany dynamicznie w zależności od stanu zamrożenia interpretera:

```
if getattr(sys, 'frozen', False):
    BASE_DIR = os.path.dirname(os.path.realpath(sys.executable))
else:
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

Taka konstrukcja izoluje wszelkie operacje wejścia/wyjścia bezpośrednio do kontekstu katalogu, w którym znajduje się uruchomiony program. Wywołanie `init()` automatycznie buduje podkatalogi (`core_data/` oraz `Reports/`) z zachowaniem restrykcyjnych uprawnień POSIX (`0o755`), chroniąc pliki baz przed nieuprawnioną modyfikacją z poziomu nieuprzywilejowanych procesów w systemie.

3. Parytet Międzyplatformowy i Mapowanie API Systemowych

Poniższa matryca architektoniczna szczegółowo opisuje, w jaki sposób system NetworkMaster zachowuje identyczny zestaw funkcji użytkowych przy jednoczesnym odpytywaniu skrajnie różnych architektur jądra:

Warstwa Funkcjonalna	Mechanizm Implementacji (Windows)	Mechanizm Implementacji (Linux)
Enumeracja Interfejsów	PowerShell <code>Get-NetAdapter</code> mapowany do obiektów przez potoki CSV.	Bezpośredni odczyt indeksów katalogów w strukturze <code>/sys/class/net/</code> .
Analityka Przepustowości	Ręczne wyliczanie różnic przesłanych bajtów w zadanym interwale czasu.	Ultrawydatne, bezprocesowe czytanie plików <code>statistics/rx_bytes</code> bezpośrednio z RAM.
Czyszczenie Cache DNS	Wywołanie systemowe procesu <code>ipconfig /flushdns</code> .	Kaskadowa sekwencja wykonawcza: <code>resolvectl</code> → <code>systemd-resolve</code> → <code>nscd</code> .
Monitoring Metryk Wi-Fi	Parsowanie strumienia wyjściowego komendy <code>netsh wlan show interfaces</code> .	Bezpośrednie odpytywanie wątków jądra za pomocą plików tekstowych w <code>/proc/net/wireless</code> .
Ekstrakcja Poświadczeń	Wyciąganie kluczy clear-text poprzez odpytywanie profili WLAN systemu.	Dostęp root i deszyfracja konfiguracji INI wewnątrz katalogu <code>/etc/NetworkManager/</code> .

4. Szczegółowa Analiza Modułów Strukturalnych ("Co do Przecinka")

4.1 Skaner Sieci Lokalnej i Odpytywanie ARP (`diag_net_scan.py`)

Tradycyjne sekwencyjne skanowanie adresów IP wprowadza potężne opóźnienia procesowe. NetworkMaster omija ten problem poprzez asynchroniczne pule wątków o wysokiej współbieżności:

- **Wielowątkowy Ping Sweep:** Wykorzystuje `concurrent.futures.ThreadPoolExecutor(max_workers=50)` do jednoczesnego rozesłania atomowych zapytań ICMP do wszystkich 254 punktów adresowych w danej masce podsieci klasy C.
- **Dynamiczna Rezolucja Prefiksu:** W systemie Linux program odnajduje aktywną trasę wyjścia komendą `ip route get 1.1.1.1 | grep -oP 'src \K\S+'` i odpowiednio ją przycina, aby zidentyfikować sieć. W systemie Windows realizowane jest to bezpośrednio przez adaptory obiektowe PowerShell.
- **Tablice Sąsiadów Jądra:** Zamiast przetwarzać powolne odpowiedzi na poziomie aplikacji, skrypt zrzuci bezpośrednio tablicę odwzorowań sprzętowych z pamięci jądra poprzez `ip neigh` (Linux) lub `arp -a` (Windows), błyskawicznie wyciągając adresy MAC i parsując je do bazy danych.

4.2 Systemy Zarządzania Interfejsami w Czasie Rzeczywistym (`net_card_menu.py`)

Ten moduł daje administratorowi pełną kontrolę nad stanem fizycznym kart sieciowych, parametrami adresacji oraz profilami połączeń:

- **Manipulacja Stanem (Toggle):** Implementuje twarde przełączanie kart (`on` / `off`). Na Linuxie wykonuje dwuetapową procedurę czyszczenia: logiczne rozłączenie przez `nmcli dev disconnect`, a następnie fizyczne wyłączenie linku sterownika przez `ip link set <iface> down`. Gwarantuje to stabilność na kartach USB i maszynach wirtualnych (VirtualBox) bez zawieszania interfejsu.
- **Automatyzacja Profili IP:** Umożliwia wstrzykiwanie konfiguracji statycznych. Konwertuje maski dziesiętne wprowadzane przez użytkownika na zapis CIDR przy użyciu przesunięć bitowych wbudowanej biblioteki `ipaddress`.

4.3 Zaawansowany Pakiet Diagnostyczno-Analityczny

- **Śledzenie Drogi Pakietów w Real-Time (`diag_tracert.py`):** Zamiast blokować pętlę interfejsu użytkownika na czas trwania długiego testu, moduł opakowuje proces systemowy w obiekt `subprocess.Popen` i podłącza się pod strumień wyjściowy, czytając go linia po linii (`stdout.readline()`). Dzięki temu użytkownik widzi każdy kolejny router na trasie natychmiast po jego odpowiedzi.
- **Benchmark Opóźnień DNS (`diag_dns_bench.py`):** Testuje jednocześnie czasy rezolucji głównych dostawców globalnych (Cloudflare, Google, Quad9). Wyciąga wartości opóźnień z komend

systemowych przy pomocy dedykowanych wyrażeń regularnych (`re`), sortuje wyniki i prezentuje je graficznie za pomocą generowanego w konsoli paska postępu.

- **Audyt Kryptograficzny SSL (`diag_ssl.py`):** Nawiązuje bezpośrednie, niskopoziomowe połączenie socketowe z wykorzystaniem kontekstu biblioteki `ssl.create_default_context()` . Łączy się z portem 443 hosta, pobiera certyfikat i wyciąga pole strukturalne `notAfter` , obliczając dokładną różnicę dni względem czasu UTC i nakładając flagi ostrzegawcze.
- **Bezpieczny WHOIS przez RDAP (`diag_whois.py`):** Zapobiega blokowaniu zapytań na klasycznym porcie 43 przez firewalles korporacyjne, wykorzystując nowoczesny protokół RDAP oparty na zapytaniach HTTPS GET. Wymusza nagłówek `User-Agent` udający przeglądarkę, eliminując limity zapytań (Rate-Limits) i zwraca czysty JSON rejestracyjny domen.

Wymóg Podniesienia Uprawnień (Privilege Escalation): Niektóre operacje diagnostyczne (takie jak czyszczenie haseł WiFi czy zmiana stanu administracyjnego kart) wymagają tokenów wysokich uprawnień (`Uruchom jako Administrator` w Windows lub kontekst `sudo` w Linux). Kod programu zabezpieczono wyjątkami, które w czytelny sposób informują użytkownika o braku uprawnień zamiast awaryjnego zamknięcia aplikacji.

5. Instrukcja Wdrożenia, Pakietowania i Kompilacji

Projekt NetworkMaster jest w pełni zoptymalizowany pod kątem automatycznego pakowania do pojedynczego pliku wykonywalnego za pomocą narzędzia **PyInstaller**, co pozwala na uruchamianie aplikacji na maszynach docelowych bez zainstalowanego interpretera Pythona.

5.1 Kompilacja w Środowisku Windows

W celu przygotowania finalnego pliku dystrybucyjnego należy uruchomić terminal CMD z uprawnieniami administratora w folderze projektu i wywołać następującą sekwencję:

```
pip install requests pyinstaller
pyinstaller --onefile --console --clean --name=NetworkMaster main.py
```

5.2 Kompilacja Binarnej Wersji Linux

Uruchom poniższy potok poleceń w celu wygenerowania samodzielnego pliku binarnego (zoptymalizowanego pod kątem dystrybucji Ubuntu, Debian, RedHat oraz Arch Linux):

```
pip3 install requests pyinstaller
pyinstaller --onefile --console --clean --name=networkmaster_linux main.py
chmod +x dist/networkmaster_linux
```

5.3 Komercyjna Lista Kontrolna Transferu (Handover)

Przekazując ten produkt nabywcy lub wystawiając go na marketplace, spakuj zasoby w następującej strukturze:

1. **Kod Źródłowy (Source):** Pełne drzewo skryptów zawierające dokładnie zweryfikowane 25 modułów Windows oraz 25 modułów Linux.
2. **Gotowe Pliki Wykonywalne (Builds):** Skompilowane uprzednio wersje binarne z katalogu `dist/` dla obu systemów operacyjnych.
3. **Czysta Struktura Danych:** Folder `core_data/` pozbawiony flagi instalacyjnej oraz jakichkolwiek logów i wpisów testowych w plikach baz danych (`.sys` / `.dat`).